Analysis of an Exact FHE Scheme Based on FFT

Cryptographic Research Log

September 17, 2026

1 Introduction and Parameters

The proposed encryption system combines concepts from signal processing (Discrete Fourier Transform, FFT) with group theory and the Chinese Remainder Theorem (CRT). It aims to perform homomorphic operations (addition and multiplication) on arrays without the ciphertext (especially the denominator) growing over time.

Global Parameters:

- **N**: The length of the plaintext arrays.
- **M**: The modulus for all value computations, where $\phi(M)|N$ must hold.
- **Z**: A sufficiently large random number.
- **G**: A multiplicative group with order ord(G) (NZ).

Tag Notation: An exponent is defined using the Chinese Remainder Theorem. The notation $[a, b] \pmod{N, Z}$ represents a unique number x for which the following holds:

$$x \equiv a \pmod{N} \tag{1}$$
$$x \equiv b \pmod{Z} \tag{2}$$

2 Key Generation and Encryption

2.1 Plaintext Preparation (Fractional Encoding)

Let the plaintext be an array of N values: $m = [m_1, m_2, \ldots, m_N]$. All mathematical operations on the values take place modulo M.

To mask the plaintext, a random array $t = [r_1, r_2, \ldots, r_N]$ is chosen. A second array s is calculated as a component-wise product:

$$s = [m_1 \cdot r_1 \pmod{M}, \ldots, m_N \cdot r_N \pmod{M}] \tag{3}$$

This allows the plaintext to be reconstructed component-wise via division (multiplication with the modular inverse):

$$m_i = s_i \cdot t_i^{-1} \pmod{M} \tag{4}$$

2.2 Transformation and Masking (Encryption)

1. **FFT**: The Discrete Fourier Transform (FFT) is applied to both arrays to obtain the encoded plaintexts:
$$\hat{s} = \mathrm{FFT}(s), \quad \hat{t} = \mathrm{FFT}(t)$$

2. **Tagging**: A group tag is appended to each element of the transformed arrays. For an index $i \in \{0, \ldots, N-1\}$ and a random value r chosen for the encryption process, the resulting sets are:
$$E_{\hat{s}} = \{(\hat{s}_i, g^{[i,r]})\}_{i=0}^{N-1}, \quad E_{\hat{t}} = \{(\hat{t}_i, g^{[i,r]})\}_{i=0}^{N-1}$$

3. **Permutation**: The order of the elements in the sets is obfuscated by a random permutation. The final ciphertext C consists of:
$$C = (\pi_s(E_{\hat{s}}), \pi_t(E_{\hat{t}}))$$

3 Homomorphic Evaluation

The system leverages the properties of the Fourier Transform: A component-wise multiplication in the time domain (plaintext) corresponds to a cyclic convolution in the frequency domain (ciphertext), and an addition remains an addition.

3.1 Rules of Fractions

Since the plaintext is fractionally represented as $m = s/t$, the classic rules of fractions apply for homomorphic operations between two ciphertexts $A = (s_A, t_A)$ and $B = (s_B, t_B)$:

- **Addition**: $\frac{s_A}{t_A} + \frac{s_B}{t_B} = \frac{s_A t_B + s_B t_A}{t_A t_B}$
- **Multiplication**: $\frac{s_A}{t_A} \cdot \frac{s_B}{t_B} = \frac{s_A s_B}{t_A t_B}$

3.2 Computing the Convolution via Tags

To compute the term $s_A \cdot s_B$ (multiplication of the values in the plaintext), for example, the evaluator must compute a cyclic convolution $\hat{s}_A * \hat{s}_B$. Since the arrays are permuted, the evaluator does not know the indices. Instead, they use the tags:
$$g^{[i,r_A]} \cdot g^{[j,r_B]} = g^{[i+j \pmod N, r_A + r_B \pmod Z]}$$

The evaluator forms all $N \times N$ cross-products of the tuples, multiplies the values and the tags, and then groups all elements that have the exact same resulting tag. The sum of the values within each group forms the exact result of the convolution.

Because all computations are strictly performed modulo M and modulo N or Z, the values never grow. No bootstrapping is required.